



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251230
Nama Lengkap	Okky Alexander
Minggu ke / Materi	15 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI PENGERTIAN REKURSIF

Fungsi rekursif merupakan sebuah fungsi dalam pemrograman yang di dalam tubuhnya terdapat pemanggilan terhadap fungsi itu sendiri. Dengan kata lain, fungsi ini mendefinisikan dirinya sendiri secara berulang dengan cara memanggil namanya kembali di dalam blok kodenya. Konsep rekursif sejatinya berasal dari dunia matematika, di mana suatu fungsi matematis dapat didefinisikan dengan mengacu pada definisi fungsi itu sendiri dalam bentuk yang lebih sederhana.

Dalam penerapannya, fungsi rekursif harus digunakan secara hati-hati karena berpotensi menyebabkan perulangan yang tidak terbatas atau infinite loop apabila tidak dirancang dengan benar. Kondisi ini dapat menghabiskan seluruh kapasitas memori yang tersedia dan menyebabkan program berhenti bekerja atau hang up. Oleh karena itu, setiap fungsi rekursif wajib memiliki dua blok penting yang bekerja secara sinergis, yaitu blok kondisi berhenti dan blok pemanggilan diri sendiri.

Dua bagian utama yang harus selalu ada dalam sebuah fungsi rekursif adalah Base Case dan Recursive Case. Base Case merupakan kondisi yang menjadi titik berhenti dari seluruh proses rekursif. Tanpa Base Case, fungsi akan terus memanggil dirinya sendiri tanpa henti. Sementara itu, Recursive Case adalah bagian yang memuat pernyataan pemanggilan fungsi itu sendiri secara berulang, dan akan terus dieksekusi hingga kondisi Base Case terpenuhi. Bentuk umum struktur fungsi rekursif dalam Python dapat dilihat pada Gambar 15.1 berikut.

```
def function_name ( parameter_list ) :  
    # Base Case: kondisi berhenti  
    if kondisi_berhenti :  
        return nilai_dasar  
    # Recursive Case: fungsi memanggil dirinya sendiri  
    return function_name ( parameter_yang_lebih_kecil )
```

Gambar 15.1 Bentuk umum struktur fungsi rekursif dalam Python

MATERI KELEBIHAN DAN KEKURANGAN FUNGSI REKURSIF

Sebagai salah satu pendekatan pemrograman yang unik, fungsi rekursif memiliki sejumlah kelebihan dan kekurangan yang perlu dipahami oleh programmer sebelum memutuskan untuk

menggunakannya. Pemahaman yang baik terhadap kedua sisi ini akan membantu dalam memilih pendekatan yang paling tepat untuk menyelesaikan suatu permasalahan.

Dari sisi kelebihan, fungsi rekursif mampu menghasilkan kode program yang jauh lebih singkat, ringkas, dan elegan dibandingkan pendekatan iteratif untuk permasalahan tertentu. Kode yang lebih pendek tidak hanya membuat program lebih mudah dibaca, tetapi juga lebih mudah dipahami alur logikanya secara keseluruhan. Selain itu, fungsi rekursif sangat efektif dalam memecah permasalahan yang kompleks menjadi sub-permasalahan yang lebih kecil dan sederhana, sehingga solusi dapat ditemukan secara bertahap dan sistematis.

Di sisi lain, fungsi rekursif juga memiliki beberapa kelemahan yang cukup signifikan. Pertama, penggunaan memori yang lebih besar menjadi konsekuensi utama karena setiap kali fungsi memanggil dirinya sendiri, sistem perlu mengalokasikan ruang memori tambahan untuk menyimpan konteks pemanggilan tersebut. Kedua, pendekatan rekursif cenderung mengorbankan efisiensi dan kecepatan eksekusi dibandingkan pendekatan iteratif yang setara. Ketiga, fungsi rekursif seringkali lebih sulit untuk dilakukan proses debugging dan kadang membutuhkan pemahaman yang lebih mendalam untuk bisa dimengerti cara kerjanya.

MATERI BENTUK UMUM DAN STUDI KASUS

Perlu dipahami bahwa pada dasarnya setiap fungsi rekursif pasti memiliki padanannya dalam bentuk solusi iteratif, begitu pula sebaliknya. Pemilihan antara kedua pendekatan ini bergantung pada konteks permasalahan dan preferensi programmer. Salah satu studi kasus yang paling sering digunakan untuk mengilustrasikan konsep rekursif adalah perhitungan faktorial.

Faktorial merupakan operasi matematika yang menghitung hasil perkalian berurutan dari seluruh bilangan bulat positif mulai dari 1 hingga nilai n yang diberikan. Secara matematis, faktorial dari n dituliskan sebagai $n!$ dan dihitung dengan rumus $1 \times 2 \times 3 \times \dots \times n$. Secara rekursif, faktorial dapat didefinisikan sebagai berikut: apabila nilai n sama dengan nol, maka hasilnya adalah 1 (Base Case); dan apabila n lebih besar dari nol, maka hasilnya adalah n dikalikan dengan faktorial dari $n-1$ (Recursive Case). Implementasi fungsi faktorial secara rekursif dalam Python dapat dilihat pada Gambar 15.2.

```
def faktorial (n):
    # Base Case: jika n = 0 atau n = 1, kembalikan 1
    if n == 0 or n == 1:
        return 1
    # Recursive Case: n * faktorial(n-1)
    else :
        return faktorial (n - 1) * n

# Contoh pemanggilan fungsi
print ( faktorial (4) )    # Output: 24
```

Gambar 15.2 Implementasi fungsi faktorial secara rekursif dalam Python

Sebagai contoh konkret, pemanggilan faktorial(4) akan menghasilkan serangkaian pemanggilan rekursif secara berantai. Dimulai dari faktorial(4) yang memanggil faktorial(3), lalu faktorial(3) memanggil faktorial(2), kemudian faktorial(2) memanggil faktorial(1). Pada saat faktorial(1) dipanggil, Base Case terpenuhi sehingga fungsi mengembalikan nilai 1. Nilai ini kemudian diteruskan kembali ke setiap pemanggilan sebelumnya secara berurutan hingga akhirnya menghasilkan nilai $4 \times 3 \times 2 \times 1 = 24$ sebagai hasil akhir.

MATERI STUDI KASUS: PERKALIAN REKURSIF

Selain faktorial, konsep rekursif juga dapat diterapkan untuk melakukan operasi perkalian antara dua buah bilangan tanpa menggunakan operator perkalian secara langsung. Ide dasarnya adalah bahwa perkalian $a \times b$ dapat dipandang sebagai penjumlahan berulang dari a sebanyak b kali. Misalnya, 2×4 dapat dihitung sebagai $2 + 2 + 2 + 2 = 8$. Dengan pola ini, Base Case terjadi ketika $bil2$ bernilai 1, di mana hasil perkaliannya adalah $bil1$ itu sendiri. Sementara Recursive Case-nya adalah $bil1$ ditambahkan dengan hasil pemanggilan perkalian($bil1$, $bil2-1$). Implementasi lengkapnya dapat dilihat pada Gambar 15.3 berikut.

```

def perkalian (bil1 , bil2 ):

    # Base Case: bil2 = 1, kembalikan bil1

    if bil2 == 1 :

        print ( '%d = ' % (bil1 ) , end = '' )

        return bil1

    # Recursive Case: bil1 + perkalian(bil1, bil2-1)

    else :

        print ( '%d + ' % (bil1 ) , end = '' )

        return bil1 + perkalian ( bil1 , bil2 - 1 )

print ( perkalian ( 2 , 4 ) ) # Output: 2 + 2 + 2 + 2 = 8

```

Gambar 15.3 Implementasi fungsi perkalian dua bilangan secara rekursif

MATERI STUDI KASUS: DERET FIBONACCI

Deret Fibonacci merupakan salah satu contoh klasik yang sangat cocok diselesaikan dengan pendekatan rekursif. Deret ini terbentuk dari penjumlahan dua bilangan sebelumnya, sehingga menghasilkan pola: 1, 1, 2, 3, 5, 8, 15, 21, 34, dan seterusnya. Secara rekursif, fungsi Fibonacci didefinisikan dengan Base Case ketika n bernilai 1 atau 2, di mana hasilnya selalu 1. Untuk nilai n yang lebih besar, Recursive Case-nya adalah $\text{fibo}(n-1)$ ditambah $\text{fibo}(n-2)$. Meskipun implementasi rekursif Fibonacci sangat elegan dan mudah dipahami, perlu diperhatikan bahwa pendekatannya bisa menjadi kurang efisien untuk nilai n yang besar karena banyak pemanggilan yang berulang. Implementasinya dapat dilihat pada Gambar 15.4.

```

fibonacci_rekursif.py

1 def fibo (n):
2     # Base Case: n = 1 atau n = 2, kembalikan 1
3     if n == 1 or n == 2:
4         return 1
5     # Recursive Case: fibo(n-1) + fibo(n-2)
6     else :
7         return fibo (n - 1) + fibo (n - 2)
8
9     # Contoh: menampilkan bilangan fibonacci ke-7
10    print ( fibo (7)) # Output: 13

```

Gambar 15.4 Implementasi fungsi deret Fibonacci secara rekursif

MATERI STUDI KASUS: KONVERSI BASIS BILANGAN

Konversi bilangan dari basis 10 ke basis lainnya seperti basis 2 (biner), basis 8 (oktal), maupun basis 16 (heksadesimal) juga dapat diselesaikan secara rekursif dengan sangat efisien. Prinsip kerjanya adalah dengan membagi bilangan secara berulang menggunakan nilai basis yang ditentukan, kemudian menyimpan sisa hasil bagi di setiap langkahnya. Base Case terjadi ketika nilai n sudah lebih kecil dari nilai basis, sehingga cukup dikembalikan karakter yang sesuai dari string referensi. Recursive Case-nya adalah dengan memanggil `toBasis(n // base, base)` dan menggabungkannya dengan karakter dari sisa bagi $n \% base$. Implementasi lengkapnya ditampilkan pada Gambar 15.5.

```
def toBasis ( n , base ) :  
    # String referensi untuk semua digit basis bilangan  
    convertString      = '0123456789ABCDEF'  
  
    # Base Case: n lebih kecil dari basis  
    if n < base :  
        return convertString [ n ]  
  
    # Recursive Case: bagi n dengan basis, ambil sisa  
    else :  
        return toBasis ( n // base , base ) + convertString [ n % base ]  
  
angka = int ( input ( 'Bilangan : '      ) )  
basis  = int ( input ( 'Basis (2/8/16) : ' ) )  
print ( toBasis ( angka , basis ) )      # Contoh: 9 basis 8 → 11
```

Gambar 15.5 Implementasi fungsi konversi basis bilangan secara rekursif

MATERI STUDI KASUS: PERMUTASI

Permutasi adalah konsep matematika yang digunakan untuk menghitung banyaknya cara menyusun r elemen dari sekumpulan n elemen yang tersedia, di mana urutan penyusunannya diperhatikan. Rumus permutasi secara matematis dinyatakan sebagai $P(n,r) = n! / (n-r)!$. Secara rekursif, pola perhitungan permutasi $P(m,n)$ dapat dirumuskan sebagai $P(m,n) = P(m, n-1) \times (m-n+1)$, dengan Base Case ketika n bernilai 0 di mana hasilnya adalah 1. Pendekatan rekursif ini memanfaatkan hubungan antara permutasi berukuran lebih besar dengan permutasi berukuran lebih kecil yang sudah dihitung sebelumnya. Implementasi fungsi permutasi secara rekursif dalam Python dapat dilihat pada Gambar 15.6.

```
def permutasi (m, n):
    # m: batas atas, n: batas bawah (m > n)

    # Base Case: jika n = 0, kembalikan 1

    if n == 0:
        return 1

    # Recursive Case: panggil permutasi(m, n-1) * (m-n+1)

    else :
        return permutasi (m, n - 1) * (m - n + 1)

# Contoh: permutasi P(10, 4)

print (permutasi (10 , 4)) # Output: 5040
```

Gambar 15.6 Implementasi fungsi permutasi secara rekursif

Dari berbagai studi kasus yang telah dibahas, dapat disimpulkan bahwa fungsi rekursif merupakan pendekatan pemrograman yang sangat powerful apabila diterapkan pada permasalahan yang memiliki struktur berulang dan dapat dipecah menjadi sub-permasalahan yang lebih kecil. Kunci keberhasilan implementasi rekursif terletak pada penentuan Base Case yang tepat dan Recursive Case yang senantiasa mengarah menuju kondisi berhenti tersebut.

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 15.1

```
def kom(n, r):  
    if r == 0:  
        return 1  
    if r == n:  
        return 1  
    return kom(n - 1, r - 1) + kom(n - 1, r)  
  
n = int(input("Masukkan n: "))  
r = int(input("Masukkan r: "))  
  
if r > n:  
    print("Error: r tidak boleh lebih besar dari n!")  
else:  
    hasil = kom(n, r)  
    print(f"C({n}, {r}) = {hasil}")
```

Gambar 15.7 Code Prima

Latihan 15.1 meminta kita untuk membuat program yang dapat mengecek apakah suatu bilangan termasuk bilangan prima atau bukan, dengan menggunakan pendekatan fungsi rekursif. Bilangan prima adalah bilangan yang hanya bisa dibagi oleh 1 dan dirinya sendiri, seperti 2, 3, 5, 7, 11, dan seterusnya.

Strategi rekursif yang digunakan adalah dengan mencoba membagi bilangan n menggunakan pembagi yang dimulai dari angka 2 dan terus bertambah satu per satu. Terdapat tiga base case dalam fungsi ini. Pertama, jika bilangan kurang dari atau sama dengan 1, langsung dikembalikan nilai False karena bilangan tersebut bukan prima. Kedua, jika nilai pembagi dikuadratkan sudah melebihi n , maka tidak ada pembagi yang cocok dan bilangan dipastikan prima sehingga dikembalikan nilai True. Ketiga, jika n habis dibagi oleh pembagi saat ini, maka bilangan tersebut bukan prima dan dikembalikan nilai False.

Jika tidak ada satu pun base case yang terpenuhi, fungsi memasuki rekursif case dengan memanggil dirinya sendiri menggunakan pembagi yang dinaikkan satu angka. Penggunaan kondisi pembagi dikuadratkan lebih dari n membuat program jauh lebih efisien karena tidak perlu mencoba semua angka hingga n , melainkan hanya sampai akar dari n .

LATIHAN 15.2

```
def cek(kalimat):  
    kalimat = kalimat.replace(" ", "").lower()  
    return pali(kalimat)  
  
def pali(s):  
  
    if len(s) <= 1:  
        return True  
  
    if s[0] != s[-1]:  
        return False  
  
    return pali(s[1:-1])  
  
kalimat = input("Masukkan kalimat: ")  
if cek(kalimat):  
    print(f'"{kalimat}" adalah palindrom')  
else:  
    print(f'"{kalimat}" bukan palindrom')
```

Gambar 15.8 Code Palidrom

Latihan 15.2 menantang kita untuk membuat fungsi rekursif yang dapat menentukan apakah sebuah kalimat merupakan palindrom atau bukan. Palindrom adalah kalimat atau kata yang bila dibaca dari depan maupun dari belakang menghasilkan bunyi yang sama, seperti kata "radar" atau kalimat "katak katak".

Sebelum masuk ke fungsi rekursif utama, string terlebih dahulu dibersihkan dari spasi dan dikonversi ke huruf kecil (lowercase) agar pengecekan tidak terpengaruh oleh perbedaan huruf besar kecil maupun spasi. Proses rekursif kemudian bekerja dengan membandingkan karakter pertama dan karakter terakhir dari string. Jika keduanya berbeda, fungsi langsung mengembalikan False karena string pasti bukan palindrom.

Base case terjadi ketika panjang string sudah 0 atau 1, yang berarti semua karakter sudah berhasil dicocokkan dan string adalah palindrom, sehingga dikembalikan nilai True. Jika karakter pertama dan

terakhir sama, fungsi memanggil dirinya sendiri dengan mengirimkan string yang sudah dipotong satu karakter dari depan dan satu dari belakang. Pola ini terus berulang hingga mencapai base case, sehingga secara bertahap mengupas lapisan string dari luar ke dalam.

LATIHAN 15.3

```
def suku(k):  
    return (2 ** k) - 1  
  
def drt(k):  
    if k == 1:  
        return 1  
    return suku(k) + drt(k - 1)  
  
n = int(input("Masukkan jumlah suku deret (n): "))  
  
print("Deret: ", end="")  
for i in range(1, n + 1):  
    if i < n:  
        print(suku(i), end=" + ")  
    else:  
        print(suku(i), end="")  
  
print(f" = {drt(n)}")
```

Gambar 15.9 Code suku deret

Latihan 15.3 meminta kita menghitung jumlah deret ganjil $1 + 3 + 7 + 15 + 31 + \dots$ sampai suku ke- n secara rekursif. Langkah pertama yang perlu dilakukan adalah memahami pola dari deret tersebut. Setiap suku ke- k pada deret ini mengikuti rumus $2^k - 1$, sehingga suku pertama adalah 1, suku kedua adalah 3, suku ketiga adalah 7, suku keempat adalah 15, dan seterusnya.

Program dibagi menjadi dua fungsi. Fungsi pertama bernama `suku_ke(k)` bertugas menghitung nilai suku ke- k menggunakan rumus $2^k - 1$. Fungsi kedua bernama `deret_ganjil(k)` adalah fungsi rekursif utama yang menjumlahkan semua suku dari suku pertama hingga suku ke- k . Base case dari fungsi rekursif ini adalah ketika k bernilai 1, maka langsung dikembalikan nilai 1 karena suku pertama deret ini

adalah 1. Pada rekursif case, fungsi mengembalikan nilai suku ke-k ditambah dengan hasil pemanggilan dirinya sendiri untuk suku ke-(k-1). Program juga menampilkan proses penjumlahan secara lengkap agar hasilnya mudah dibaca, misalnya menampilkan "1 + 3 + 7 + 15 + 31 = 57" untuk input n = 5.

LATIHAN 15.4

```
def jumladgt(n):  
    n = abs(n)  
    if n < 10:  
        return n  
    return (n % 10) + jumladgt(n // 10)  
  
bilangan = int(input("Masukkan bilangan: "))  
  
digits = [int(d) for d in str(abs(bilangan))]  
proses = " + ".join(map(str, digits))  
hasil = jumladgt(bilangan)  
  
print(f"Jumlah digit dari {bilangan}: {proses} = {hasil}")
```

Gambar 15.10 Code Jumlah Digit

Latihan 15.4 meminta kita membuat fungsi rekursif untuk menghitung jumlah seluruh digit dari sebuah bilangan. Sebagai contoh, bilangan 234 memiliki jumlah digit $2 + 3 + 4 = 9$ sesuai yang disebutkan pada soal.

Ide utama dari solusi rekursif ini adalah dengan memanfaatkan operasi modulo dan pembagian integer. Digit paling kanan (digit terakhir) dari sebuah bilangan bisa diambil menggunakan operasi $n \% 10$, sedangkan sisa bilangan setelah digit terakhir dibuang bisa diperoleh menggunakan $n // 10$. Base case terjadi ketika bilangan sudah kurang dari 10, artinya hanya tersisa satu digit, sehingga nilainya langsung dikembalikan.

Pada rekursif case, fungsi mengembalikan digit terakhir ($n \% 10$) ditambah dengan hasil pemanggilan fungsi untuk sisa bilangan ($n // 10$). Misalnya untuk bilangan 234, fungsi pertama mengambil digit 4, lalu memanggil dirinya dengan bilangan 23, kemudian mengambil digit 3 dan memanggil dirinya dengan bilangan 2, lalu karena 2 kurang dari 10 maka base case terpenuhi dan dikembalikan 2. Hasilnya adalah $4 + 3 + 2 = 9$.

LATIHAN 15.5

```
def komb(n, r):  
    if r == 0:  
        return 1  
    if r == n:  
        return 1  
    return komb(n - 1, r - 1) + komb(n - 1, r)  
  
n = int(input("Masukkan n: "))  
r = int(input("Masukkan r: "))  
  
if r > n:  
    print("Error: r tidak boleh lebih besar dari n!")  
else:  
    hasil = komb(n, r)  
    print(f"C({n}, {r}) = {hasil}")
```

Gambar 15.11 Code Hitung kombinasi

Latihan 15.5 meminta kita untuk menghitung kombinasi $C(n, r)$ secara rekursif. Kombinasi adalah cara menghitung banyaknya cara memilih r elemen dari n elemen tanpa memperhatikan urutan pemilihannya. Rumus kombinasi secara matematis adalah $C(n, r) = n! / (r! * (n-r)!)$.

Pendekatan rekursif yang digunakan dalam program ini memanfaatkan identitas Segitiga Pascal, yaitu $C(n, r) = C(n-1, r-1) + C(n-1, r)$. Identitas ini menyatakan bahwa jumlah cara memilih r elemen dari n elemen sama dengan penjumlahan dua sub-masalah yang lebih kecil, yaitu kasus di mana elemen tertentu dipilih $C(n-1, r-1)$ dan kasus di mana elemen tersebut tidak dipilih $C(n-1, r)$.

Terdapat dua base case dalam fungsi ini. Pertama, $C(n, 0) = 1$ karena hanya ada satu cara untuk memilih 0 elemen dari n elemen, yaitu tidak memilih apapun. Kedua, $C(n, n) = 1$ karena hanya ada satu cara untuk memilih semua n elemen sekaligus. Selain kedua kondisi tersebut, fungsi akan memanggil dirinya sendiri secara rekursif mengikuti pola Segitiga Pascal hingga akhirnya mencapai salah satu base case. Sebagai contoh, $C(5, 2)$ akan menghasilkan nilai 10 yang berarti ada 10 cara berbeda untuk memilih 2 elemen dari 5 elemen yang tersedia.